Functions

CSL100

August 29, 2024

Quick Overview

- We've already learned
  - C data types
  - Operations, expressions and statements
  - Conditional statements
  - Loops
- Mid-sem additional material
  - Functions
  - Arrays

This Class

- Introduction to functions
  - Basic syntax
  - Basic use cases

A Modern Smartphone



- Surf the net
  - Input: Web address
  - Output: Desired page
- Book tickets
  - Input: userid, password, booking info, bank info
  - Output: Ticket
- Send email
  - Input: email address of receiver, mail text
  - Output: --
- Take photos
  - Input: --
  - Output: Picture
- Talk (we can do that too!!)
  - Input: Phone number
  - Output: Conversation (if lucky)
- …

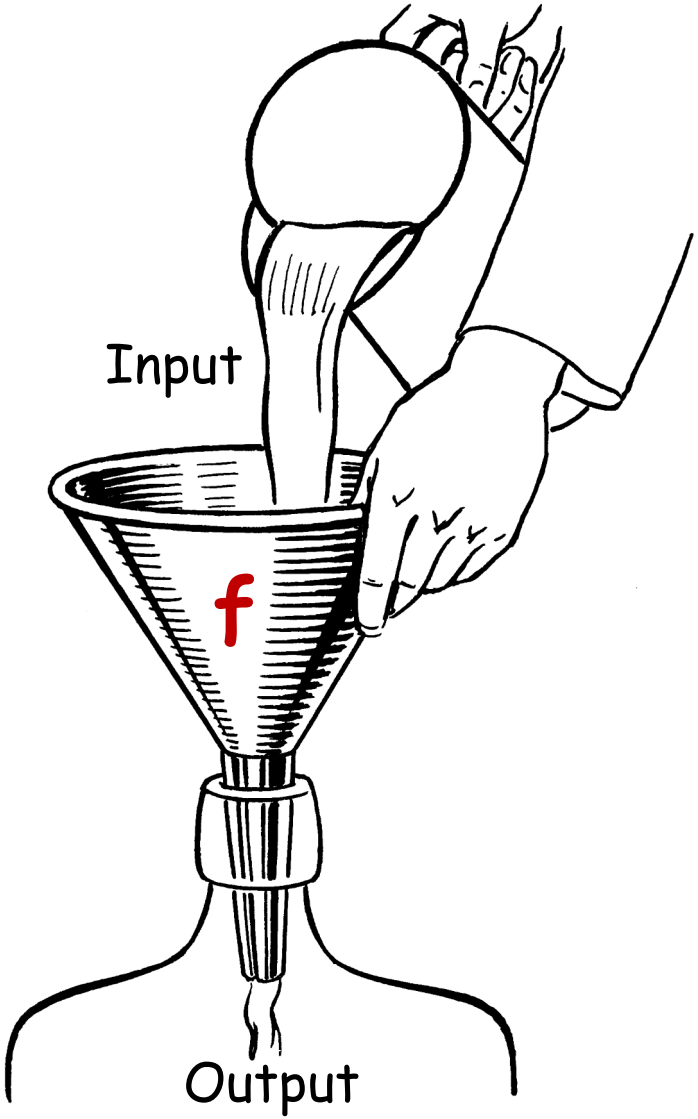
Lots of Related/Unrelated Task to Perform

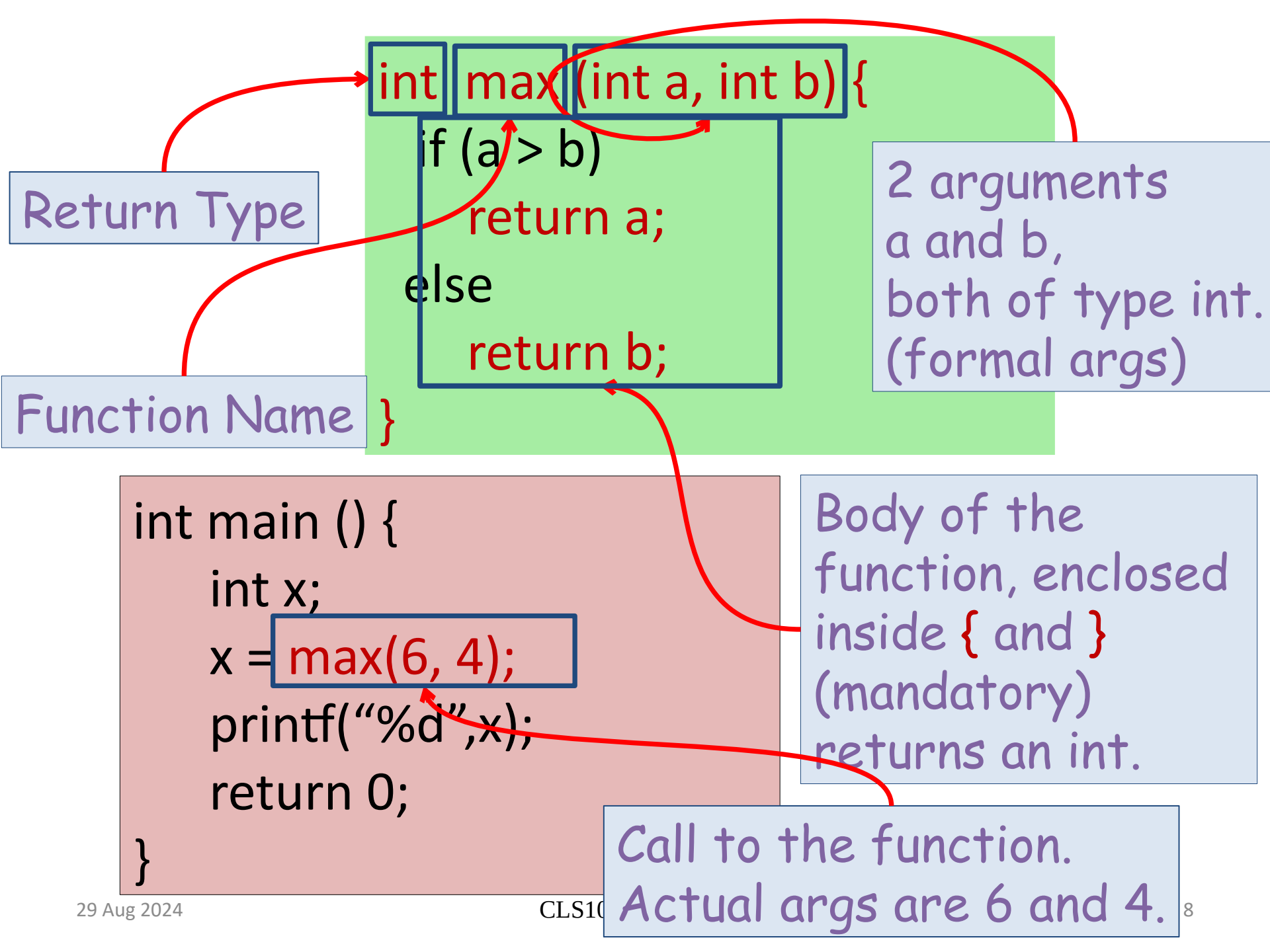
- Divide and Conquer
  - Create well defined sub tasks
  - Work on each task independently
    - Development, Enhancements, Debugging
- Reuse of tasks
  - Email and Chat apps can share spell checker
  - Phone and SMS apps can share dialer
- C facilitates this using Functions

Function

- An independent, self-contained entity of a C program that performs a well-defined task
- It has
  - Name: for identification
  - Arguments: to pass information from outside world (rest of the program)
  - Body: processes the arguments do something useful
  - Return value: To communicate back to outside world
    - Sometimes not required

Parts of a Function




int max (int a, int b) {
if (a > b)
return a;
else
return b;
}
Return Type
Function Name
2 arguments
a and b,
both of type int.
(formal args)
int main () {
int x;
x = max(6, 4);
printf("%d",x);
return 0;
}
Body of the
function, enclosed
inside { and }
(mandatory)
returns an int.
Call to the function.
Actual args are 6 and 4.

Tracing the code

```
int  max (int a, int b) {
   if (a > b)
       return a;
  else
       return b;
}
```

```
int main () {
    int x;
    x = max(6, 4);
    printf("%d",x);
    return 0;
}
```

Tracing the code

```
int  max (int a, int b) {
    if (a > b)
        return a;
    else
        return b;
}
```

Assigned a=6,b=4

```
int main () {
    int x;
    x = max(6, 4);
    printf("%d",x);
    return 0;
}
```

Execution jumps to max

Tracing the code

```
int  max (int a, int b) {
    if (a > b)
        return a;
    else
        return b;
}
```

6>4 → True

```
int main () {
    int x;
    x = max(6, 4);
    printf("%d",x);
    return 0;
}
```

Execution jumps to max

Tracing the code

```
int  max (int a, int b) {
    if (a > b)
        return a;
    else
        return b;
}
```

Returns 6

```
int main () {
    int x;
    x = max(6, 4);
    printf("%d",x);
    return 0;
}
```

Execution jumps to max

Tracing the code

```
int  max (int a, int b) {
    if (a > b)
        return a;
    else
        return b;
}
```

```
int main () {
    int x;
    x = max(6, 4);
    printf("%d",x);
    return 0;
}
```

x= 6

Execution jumps to max

Tracing the code

```
int  max (int a, int b) {
    if (a > b)
        return a;
    else
        return b;
}
```

```
int main () {
    int x;
    x = max(6, 4);
    printf("%d",x);
    return 0;
}
```

output 6

Execution jumps to max

Arguments

- Input to the function
  - Should have matching type
  - Type should be declared

- A new copy of these arguments is made
  - Function works on these new copies

Why use functions?

Example : Maximum of 3 numbers

```
int main(){
   int a, b, c, m;

   /* code to read
    * a, b, c */

   if (a>b){
      if (a>c) m = a;
      else m = c;
   }
   else{
      if (b>c) m = b;
      else m = c;
   }

   /* print or use m */

   return 0;
}
```

```
int max(int a, int b){
   if (a>b)
     return a;
   else
     return b;
}

int main() {
   int a, b, c, m;

   /* code to read
    * a, b, c */

   m = max(a, b);
   m = max(m, c);
   /* print or use m */

   return 0;
}
```

This code can scale easily to handle large number of inputs (e.g.: max of 100 numbers!)

A=5,b=23,c=19
m=23=max(5,23)
m=max(23,19)
m=23

Why use Functions?

- Break up complex problem into small sub-problems
- Solve each of the sub-problems separately as a function, and combine them together in another function
- The main tool in C for modular programming

Advantages of Using Functions

- **Code Reuse**: Allows us to reuse a piece of code as many times as we want, without having to write it
  - Think of the `printf` function!
- **Procedural Abstraction**: Different pieces of your algorithm can be implemented using different functions
- **Distribution of Tasks**: A large project can be broken into components and distributed to multiple people
- **Easier to debug**: If your task is divided into smaller subtasks, it is easier to find errors
- **Easier to understand**: Code is better organized and hence easier for an outsider to understand it

We Have Seen Functions Before

- main() is a special function. Execution of program starts from the beginning of main().
- scanf(…), printf(…) are standard input-output library functions.
- sqrt(…), pow(…) are math functions in math.h

Function Call

- A function call is an *expression*
  - feeds the necessary values to the function arguments
  - directs a function to perform its task
  - receives the return value of the function
- Similar to operator application

5 + 3 is an expression of type integer that evaluates to 8

max(5, 3) is an expression of type integer that evaluates to 5

Function Call

- Since a function call is an *expression*
  - it can be used anywhere an expression can be used
  - subject to type restrictions

```
printf("%d", max(5,3));
max(5,3) – min(5,3)
max(x, max(y, z)) == z

if (max(a, b)) printf("Y");
```

prints 5

evaluates to 2

checks if z is max of x, y, z

prints Y if max of a and b is not 0.

Returning from a function: Type

- Return type of a function tells the type of the result of function call
- Any valid C type
  - int, char, float, double, …
  - **void**
- Return type is void if the function is not supposed to return any value

```
void print_one_int(int n) {
    printf("%d", n);
}
```

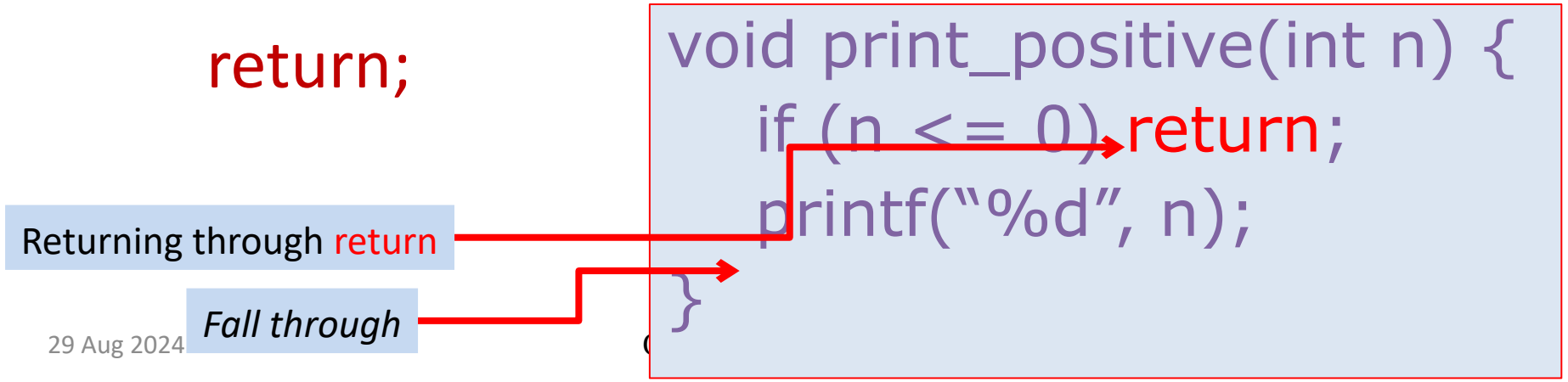
Returning from a Function: return Statement

- If return type is not void, then the function should return a value:

  return return_expr;

- If return type is void, the function may *fall through* at the end of the body or use a return without return_expr:

  return;



Returning from a Function: return Statement

- When a return statement is encountered in a function definition
  - control is immediately transferred back to the statement making the function call in the parent function
- A function in C can return only ONE value or NONE.
  - Only one return type (including void)

Function Declaration- **Prototype**

- A function declaration is a statement that tells the compiler about the different properties of that function
  - name, argument types and return type of the function
- Structure:

```
return_type function_name (list_of_args);
```

- Looks very similar to the first line of a function definition, but NOT the same
  - has semicolon at the end instead of BODY

Function Declaration

return_type function_name (list_of_args);

- Examples:
  - int max(int a, int b);
  - int max(int x, int y);
  - int max(int , int);

All 3 declarations are equivalent! Since there is no BODY here, argument names do not matter, and are optional.

- Position in program: Before the call to the function
  - allows compiler to detect inconsistencies
  - Header files (stdio.h, math.h,…) contain declarations of frequently used functions
  - #include <…> just copies the declarations

Next Class

- Functions
  - Execution trace
  - Parameter passing
  - More examples